



Maghreb University of Science and Technology (MUST)

Intelligent Engine Failure Detection and Maintenance Management Platform

Waterfall-Based PFE Report for Licence in Software Engineering and Information Systems

Capstone Internship / PFE Report

Prepared by: Ilyes Neguir

Academic supervisor: To be completed

Student ID: To be completed

Academic year: 2025-2026

Declaration

I hereby declare that the work presented in this report, entitled "Intelligent Engine Failure Detection and Maintenance Management Platform", was carried out as my PFE project for the Licence in Software Engineering and Information Systems at Maghreb University of Science and Technology (MUST). The report is based on the project source package, the existing report draft, and the technical project summary. Personal administrative details such as student ID and supervisor names should be completed with the official information before submission.

Abstract

This project presents a web platform for vehicle diagnostic booking and AI-assisted engine fault detection. Clients manage vehicles and request diagnostic appointments, while mechanics manage bookings, enter OBD readings, receive machine-learning fault predictions, record maintenance actions, and generate PDF reports. The system follows a multi-service architecture composed of a React/TypeScript frontend, a Spring Boot backend, a FastAPI machine-learning service, PostgreSQL persistence, and development support through Docker and MailHog. The report is structured using the Waterfall methodology: requirements, analysis, design, implementation, verification, and delivery. The platform demonstrates how OBD data and neural-network inference can be integrated into an operational maintenance workflow rather than remaining a standalone model.

Acknowledgments

I would like to thank my supervisors and everyone who supported this project through feedback, technical discussion, and review. This PFE was a valuable opportunity to connect software engineering, backend architecture, machine learning, and a practical automotive maintenance use case.

Contents

Declaration; Abstract; Acknowledgments; List of Abbreviations; Introduction; Project Context and Motivation; Waterfall Methodology; Requirements Analysis; Use Case Model; System Design; Database and Domain Model; OBD Parameter Dictionary; Booking Lifecycle; Diagnostic Data and ML Integration; Implementation; Module Breakdown; Report Generation; Deployment Environment; API Summary; Testing and Validation; Limitations; Problems Faced and Solutions; Conclusion and Future Work; References.

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DTO	Data Transfer Object
JWT	JSON Web Token
ML	Machine Learning
OBD	On-Board Diagnostics
PFE	Projet de Fin d'Etudes
REST	Representational State Transfer
UML	Unified Modeling Language

Introduction

This report presents the design and implementation of an Intelligent Engine Failure Detection and Maintenance Management Platform developed as a PFE project for the Licence in Software Engineering and Information Systems at MUST University.

The project combines two parts that are often separated in small workshop environments: the operational management of maintenance bookings and the technical analysis of engine sensor data. Clients can register their vehicles and request diagnostic appointments. Mechanics can accept bookings, start a diagnostic session, enter OBD readings, receive an AI-assisted fault classification, record maintenance notes, and generate a report for the client.

The report was rewritten to follow the Waterfall methodology. This is more coherent for the final academic presentation because the work can be explained through clear successive phases: requirements, analysis, design, implementation, verification, and delivery. The technical content remains based on the real project material: a React/TypeScript frontend, a Spring Boot backend, a FastAPI machine learning service, PostgreSQL persistence, Docker-based development services, and PDF reporting.

Project Context and Motivation

Traditional vehicle maintenance is usually reactive. A client reports symptoms after they become visible, the mechanic performs checks, and the diagnostic result is often stored in an informal way. This creates weak traceability between the appointment, the vehicle, the diagnostic data, the prediction result, and the final maintenance action.

Modern vehicles expose useful information through OBD systems. Parameters such as MAP, TPS, RPM, AFR, Lambda, CO, CO₂, O₂, and fuel consumption can support better fault identification when processed consistently. The project uses these readings as structured input for a trained neural network and connects the prediction to a booking workflow.

The goal is not to replace the mechanic. The platform is designed as decision support: it gives the mechanic a clear prediction and confidence score, while the mechanic remains responsible for interpreting the result and completing the maintenance record.

Project Explained in Simple Terms

The easiest way to understand the platform is to imagine a small digital garage. A client owns a vehicle and wants a diagnostic appointment. Instead of calling the mechanic and keeping notes on paper, the client uses the application to create a booking. The booking becomes the central file for that vehicle visit.

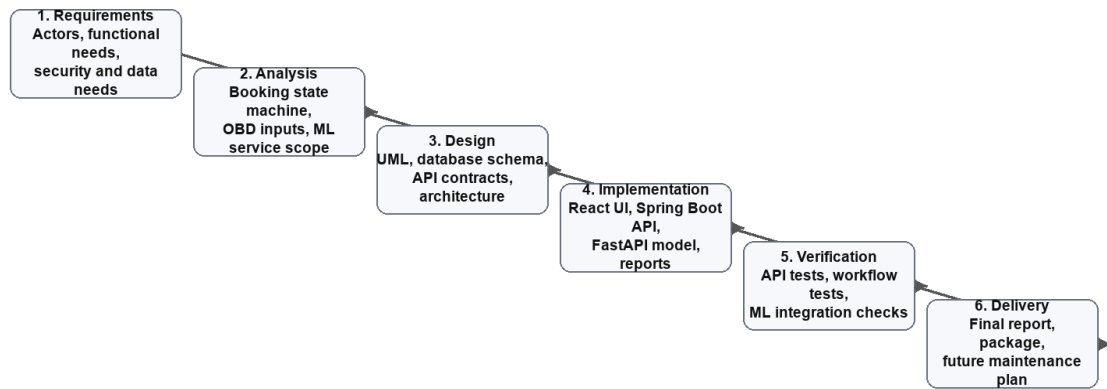
When a mechanic accepts the booking, the system knows who is responsible for the intervention. When the diagnostic starts, the mechanic enters measured OBD values. These values are not random text notes; they are numerical sensor readings that can be processed by a machine-learning model.

The AI part receives the sensor values and returns a predicted fault category with a confidence score. The mechanic can then use this result as support for the technical diagnosis. Finally, the system keeps the result, the mechanic notes, and the generated PDF report linked to the same booking.

This means the project has two connected goals. The first goal is management: users, vehicles, bookings, maintenance records, reports, and security. The second goal is intelligence: transforming OBD data into a useful fault prediction. The platform is valuable because it joins both goals in one workflow.

Waterfall Methodology

Waterfall Methodology Applied to the Project



The Waterfall model organizes the project into sequential phases. Each phase produces outputs that become the input of the next phase. For this PFE, the approach is appropriate because the final report must clearly show traceability from requirements to design, implementation, and testing.

The methodology was applied as follows. First, the requirements phase defined the actors, functional needs, security constraints, and main data objects. Second, the analysis phase clarified the booking lifecycle and the diagnostic workflow. Third, the design phase produced the architecture, database schema, UML diagrams, and API contracts. Fourth, the implementation phase realized the frontend, backend, ML service, and report generation. Fifth, the verification phase checked authentication, booking transitions, diagnostic submission, prediction persistence, and PDF generation.

The academic documentation is structured using Waterfall phases. This avoids mixing several methodologies in the same report and makes the defense easier to follow from the first requirement to the final validation.

Waterfall phase	Project output
Requirements	Stakeholders, functional requirements, non-functional requirements, use cases
Analysis	Booking states, actor permissions, OBD input list, report needs
Design	Layered architecture, UML diagrams, relational schema, REST contracts
Implementation	React interface, Spring Boot services, FastAPI inference service, PDF generation
Verification	Manual and API tests for authentication, bookings, diagnostics, reports
Delivery	Final report package, limitations, future improvements

Waterfall Phase 1: Requirements

The requirements phase answers the question: what must the system do? At this stage, the project is described from the point of view of the people who will use it. The client wants to manage vehicles, request a diagnostic appointment, and receive a readable report. The mechanic wants to see requests, accept a job, perform the diagnostic, and record the intervention.

The important decision in this phase was to make the booking the center of the system. Without the booking, the ML model would only be a separate prediction tool. With the booking, each prediction is connected to a real vehicle, a real user, a mechanic, a time, a status, and a final report.

The requirements phase also defined access rules. A client should not see another client's vehicle. A mechanic should not submit diagnostic values for a booking assigned to another mechanic. These rules are not visual details; they are part of the real business logic of the system.

Waterfall Phase 2: Analysis

The analysis phase studies the requirements and turns them into clear system behavior. For this project, the most important behavior is the booking lifecycle. A booking starts as PENDING, becomes CONFIRMED when accepted by a mechanic, becomes IN_PROGRESS when the diagnostic begins, and becomes COMPLETED when the work is finished.

The analysis phase also clarified the diagnostic flow. The mechanic does not send free text to the AI service. The mechanic submits a fixed list of 14 numeric OBD values. The backend checks that the values exist, then sends them to the FastAPI service in the expected structure. This avoids inconsistent input and makes the prediction reproducible.

Another analysis result is the separation between business logic and prediction logic. Spring Boot is responsible for users, roles, booking rules, database records, and reports. FastAPI is responsible only for loading the model and producing predictions. This separation keeps the system easier to maintain.

Waterfall Phase 3: Design

The design phase answers how the system will be built before writing the final implementation. The chosen design is a layered and service-oriented architecture. The frontend communicates with the backend through REST APIs. The backend communicates with the database through JPA repositories and with the ML service through an HTTP client.

The class and database design are based on the main entities: User, Vehicle, Booking, OBDData, MaintenanceRecord, and Report. The relationships are simple but important. A User owns Vehicles. A Booking links a Client, a Vehicle, and possibly a Mechanic. OBDData belongs to one Booking. A Report belongs to one Booking. MaintenanceRecord belongs to a Booking.

This design makes the report generation possible because all necessary data can be reached through the booking. It also makes authorization clearer because the system can check whether the authenticated user is the vehicle owner or the assigned mechanic.

Waterfall Phase 4: Implementation

The implementation phase transformed the design into working software. The frontend was implemented with React and TypeScript to build reusable screens and components. TailwindCSS was used to style the interface quickly while keeping visual consistency. The mechanic booking detail screen became one of the key screens because it brings together the booking status, diagnostic entry, prediction result, and report action.

The backend was implemented with Spring Boot. Controllers receive HTTP requests, services apply business rules, repositories access PostgreSQL, and DTOs control the shape of data exchanged with the frontend or ML service. Spring Security and JWT protect the endpoints and identify the current user.

The machine-learning service was implemented separately in Python with FastAPI. This is a practical choice because ML libraries such as Keras, scikit-learn, and joblib are more natural in the Python ecosystem. Spring Boot therefore does not need to load the neural network directly; it only sends the OBD values and receives the result.

Waterfall Phase 5: Verification

The verification phase checks whether the implementation satisfies the requirements. In this project, verification must cover both normal cases and protection rules. For example, it is not enough to test that a mechanic can submit OBD data. The system must also reject an unassigned mechanic who tries to submit data for someone else's booking.

The most important verification scenario is the complete path from booking to report. A client creates a vehicle and a booking. A mechanic accepts the booking, starts it, submits the 14 OBD values, receives the prediction, records notes, and generates the report. If this scenario works, the main value of the platform is demonstrated.

Verification also includes checking API contract details. The project experienced field-name issues between backend Java Records and frontend TypeScript usage. This shows that testing should include not only backend logic, but also whether the frontend correctly displays the response fields.

Requirements Analysis

The system has two principal actors. The Client manages personal vehicles, creates diagnostic bookings, views booking status, and downloads reports. The Mechanic manages pending and assigned bookings, starts the intervention, enters OBD values, reviews the AI prediction, completes the booking with notes, and generates reports.

The requirements are expressed as stable functional requirements. This fits the Waterfall structure because the requirement set is validated before design and implementation.

ID	Requirement	Actor	Priority
FR-01	Register and authenticate users with CLIENT or MECHANIC role.	Client, Mechanic	High
FR-02	Allow clients to create and manage their own vehicle records.	Client	High
FR-03	Allow clients to create diagnostic bookings linked to owned vehicles.	Client	High
FR-04	Allow mechanics to view pending bookings and their assigned bookings.	Mechanic	High
FR-05	Enforce booking states: PENDING, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELED.	System	High
FR-06	Allow assigned mechanics to submit 14 OBD sensor values for a booking.	Mechanic	High
FR-07	Call the FastAPI ML service and store the predicted fault and confidence score.	System	High
FR-08	Generate a PDF diagnostic report linked to a booking.	Client, Mechanic	High
FR-09	Store maintenance records including intervention, notes, parts replaced, and cost.	Mechanic	Medium
FR-10	Send development email notifications for booking confirmation and report readiness.	System	Medium

Non-Functional Requirements

The platform also requires security, reliability, maintainability, and clear data ownership. These constraints are important because diagnostic data is tied to clients, vehicles, and mechanics.

Category	Requirement
Security	Use JWT authentication and role-based access control for CLIENT and MECHANIC actions.
Data ownership	Clients can access only their own vehicles and bookings; mechanics can act only on assigned bookings for diagnostic submission.
Maintainability	Separate frontend, backend, ML service, and database responsibilities.
Traceability	Link every OBDData and Report record to a specific booking.
Usability	Provide a clear diagnostic interface where the prediction result is visible and readable.
Portability	Use Docker Compose for PostgreSQL, backend, ML service, and MailHog development support.

Use Case Model

The use case model describes what the two main actors can do in the system. It also clarifies the boundary between the business application and the machine-learning service. The client never calls the ML service directly. The mechanic uses the diagnostic feature through a booking, and the backend is responsible for calling FastAPI and storing the result.

The strongest use case is the diagnostic booking flow: a client creates a booking, a mechanic accepts it, the mechanic starts the diagnostic session, OBD values are submitted, the model returns a prediction, and the report is generated. This chain is important because it proves that the application is not only a prediction API but a complete maintenance workflow.

Actor	Main use cases
Client	Register, log in, manage vehicles, create bookings, view booking history, view diagnostic result, download report.
Mechanic	Register, log in, view pending bookings, accept booking, start booking, submit OBD data, review prediction, complete booking, create maintenance record, generate report.
Spring Boot backend	Authenticate users, enforce ownership, manage state transitions, validate DTOs, persist data, call ML service, create PDF reports.
FastAPI service	Load Keras model and scaler, receive numeric OBD payload, return predicted class and confidence.

Detailed Use Cases

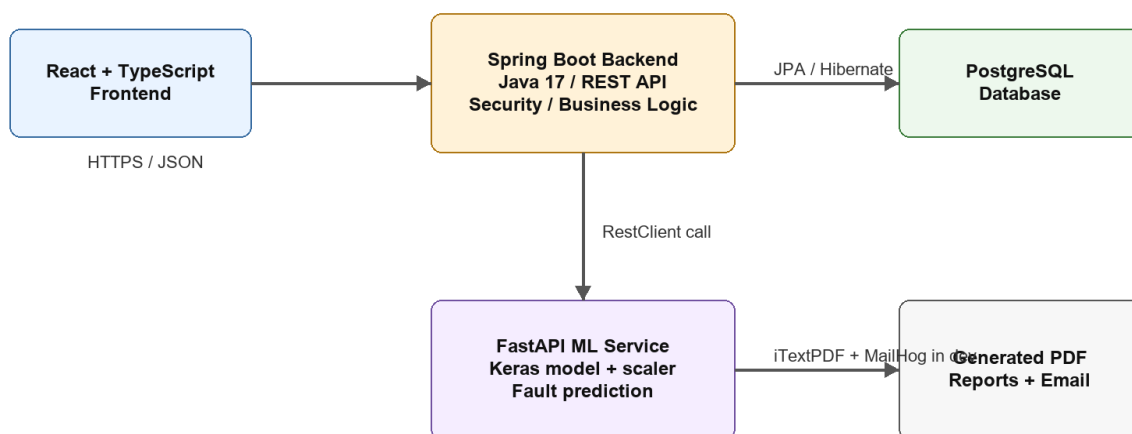
The following table gives the main use cases in a compact form suitable for a final report. Each use case includes its trigger and expected result, which helps connect requirements to design and testing.

Use case	Trigger	Expected result
Create vehicle	Client submits vehicle information.	Vehicle is stored and linked to the authenticated owner.

Use case	Trigger	Expected result
Create booking	Client selects vehicle, date/time, and description.	Booking is created with PENDING status and no assigned mechanic.
Accept booking	Mechanic accepts a pending request.	Booking becomes CONFIRMED and mechanic is assigned.
Start diagnostic	Assigned mechanic begins the work.	Booking becomes IN_PROGRESS, allowing diagnostic entry.
Submit OBD data	Mechanic submits the 14 required values.	Backend validates values, calls ML service, and stores OBDData.
Generate report	Mechanic or authorized client requests report generation.	PDF file is generated and linked to the booking.
View maintenance history	Client or mechanic requests history for an authorized vehicle/booking.	System returns previous diagnostic and maintenance records.

System Design

System Architecture



The design follows a multi-service architecture. The React/TypeScript frontend is responsible for the user interface. The Spring Boot backend is the authoritative business layer: it handles authentication, users, vehicles, bookings, diagnostics, maintenance records, reports, and security. The FastAPI service is a sidecar dedicated to machine learning inference. PostgreSQL stores persistent data.

This separation is important because the Java backend manages transactional business logic, while the Python service loads the Keras model and joblib scaler. The backend communicates with FastAPI through a REST client and stores the prediction result returned by the ML layer.

Why This Architecture Was Chosen

The project could have been built as one single application, but the chosen architecture separates responsibilities. This is useful because the backend and ML service have different needs. The backend needs strong business rules, security, and database transactions. The ML service needs Python libraries and model-loading logic.

Spring Boot is a good choice for the backend because it is structured, stable, and well suited for REST APIs, authentication, validation, and database persistence. It also fits academic software engineering expectations because it encourages a clear controller-service-repository organization.

FastAPI is a good choice for the prediction service because it is lightweight, fast, and compatible with the Python ML ecosystem. It can expose a simple HTTP endpoint while internally loading a Keras model and scaler.

React and TypeScript are suitable for the frontend because the interface has many states: pending bookings, confirmed bookings, diagnostic forms, prediction display, and reports. TypeScript helps reduce mistakes when working with API response fields.

Database and Domain Model

The main domain entities are User, Vehicle, Booking, OBDData, Report, and MaintenanceRecord. The Booking entity is central because it connects the client, mechanic, vehicle, diagnostic readings, generated report, and maintenance notes.

Entity	Important fields	Purpose
User	id, name, email, password, phone, role	Represents a CLIENT or MECHANIC account.
Vehicle	id, owner, make, model, year, vin, licensePlate	Stores vehicle identity and ownership.
Booking	client, mechanic, vehicle, scheduleTime, status, description, mechanicNotes	Controls the maintenance workflow.
OBDData	booking, 14 readings, predictedFault, confidenceScore, createdAt	Stores diagnostic input and AI output.
Report	booking, filePath, fileName, createdAt	References the generated diagnostic PDF.
MaintenanceRecord	booking, intervention, notes, partsReplaced, cost	Records work performed by the mechanic.

OBD Parameter Dictionary

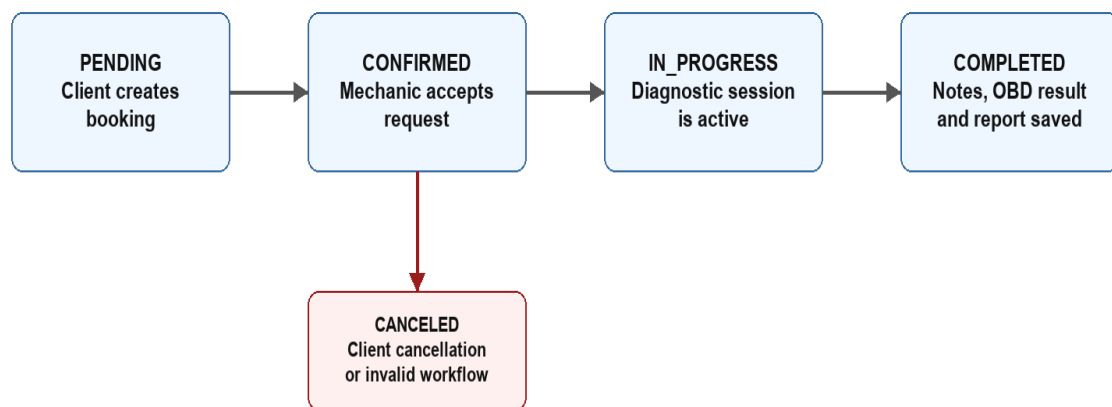
The project uses a fixed vector of 14 OBD-related numerical values. A fixed order is important because the scaler and neural network expect the same feature structure used during training. The backend DTO therefore requires all values before calling the ML service.

Parameter	Meaning in the diagnostic context
MAP	Manifold absolute pressure, useful for detecting intake and load-related anomalies.
TPS	Throttle position sensor value, indicating driver demand and engine response.
Force	Derived mechanical force value used by the model input vector.
Power	Derived power value representing engine output behavior.
RPM	Engine revolutions per minute, one of the core operating indicators.
Consumption_LH	Fuel consumption per hour.
Consumption_L100KM	Fuel consumption per 100 kilometers.
Speed	Vehicle speed during the reading.
CO	Carbon monoxide indicator related to combustion quality.
HC	Hydrocarbon emission indicator.

Parameter	Meaning in the diagnostic context
CO2	Carbon dioxide emission indicator.
O2	Oxygen reading, useful for mixture interpretation.
Lambda	Air-fuel equivalence indicator.
AFR	Air-fuel ratio, central for detecting rich or lean mixture behavior.

Booking Lifecycle

Booking and Diagnostic Lifecycle



The booking lifecycle is a strict state machine. A client creates a booking in **PENDING** state. A mechanic accepts it, which assigns the mechanic and changes the status to **CONFIRMED**. The assigned mechanic can start the work, changing the booking to **IN_PROGRESS**. Diagnostic submission is only meaningful in the active workflow because the OBD values must be linked to the correct vehicle and booking. When the mechanic completes the intervention, the booking becomes **COMPLETED** and the final notes and report are attached.

This lifecycle is one of the key differences between this project and a simple ML demo. The AI prediction is not isolated; it is embedded in a controlled maintenance process.

Diagnostic Data and ML Integration

The diagnostic module uses 14 required OBD sensor values: MAP, TPS, Force, Power, RPM, Consumption_LH, Consumption_L100KM, Speed, CO, HC, CO2, O2, Lambda, and AFR. These values are validated in the Spring Boot DTO before being sent to the FastAPI service.

The FastAPI service loads the trained Keras model and feature scaler at startup. During prediction, it receives the numerical reading, scales the features, runs inference, and returns a predicted class with a confidence percentage. The Spring Boot backend then stores the resulting fault label and confidence in `OBDData`.

A real implementation detail discovered during review is the label mapping gap. FastAPI returns detailed labels such as Normal/Baseline Operation, Rich Mixture Problems, and Combustion Efficiency Problems. The Java `DiagnosticService` maps the returned integer to its own labels: No Fault, Ignition Fault, and Fuel System/Emission Control Fault. This report documents the behavior as implemented instead of pretending the two labels are

identical.

Class	FastAPI description	Stored Java label
0	Normal/Baseline Operation	No Fault
1	Rich Mixture Problems	Ignition Fault
2	Combustion Efficiency Problems	Fuel System/Emission Control Fault

Machine Learning Flow Explained

The machine-learning part can be explained as a pipeline. First, the system receives raw numerical values from the diagnostic form. Second, the values must be arranged in the same feature order used during model training. Third, the scaler transforms the numbers so they are in the range expected by the neural network. Fourth, the Keras model produces a probability distribution over the known fault classes.

The model does not understand cars like a human mechanic. It recognizes patterns in the numerical data based on what it learned during training. For example, mixture-related values such as AFR, Lambda, O₂, CO, CO₂, and HC can help identify abnormal combustion or fuel-air balance. RPM, MAP, TPS, Force, and Power help describe the operating condition of the engine.

The confidence score expresses how strongly the model selected a class. It should be presented carefully. A high confidence does not mean the mechanic must blindly accept the result. It means the model found the submitted sensor pattern close to a learned class. The mechanic still has to compare the result with physical symptoms and professional experience.

In the implemented flow, FastAPI returns the predicted class and confidence. Spring Boot then stores the prediction with the booking. This is important because the prediction becomes part of the maintenance history, not just a temporary screen result.

Security and Access Control Explained

Security in this project is based on authentication and authorization. Authentication answers: who is the current user? Authorization answers: what is this user allowed to do? The backend uses JWT tokens so that each protected request can be connected to a logged-in user.

The CLIENT role and MECHANIC role do not have the same permissions. A client can manage personal vehicles and create bookings. A mechanic can view pending work and assigned bookings, but the mechanic must be assigned before performing sensitive diagnostic actions.

Ownership checks are especially important for vehicles. A client should not be able to create a booking for a vehicle that belongs to another account. The backend therefore checks the authenticated email or user identity against the vehicle owner before allowing actions.

These checks are implemented in the backend, not only in the frontend. This matters because frontend buttons can be hidden, but API requests can still be attempted manually. Real protection must be enforced by the server.

Implementation

The implementation uses Java 17 and Spring Boot 3.2.3 for the primary backend. Spring Security and JWT protect the API. Spring Data JPA manages database persistence. Lombok reduces boilerplate in entities and DTOs. The backend exposes REST endpoints for authentication, vehicles, bookings, diagnostics, reports, maintenance records, ML passthrough testing, and email actions.

The frontend is a React/TypeScript application using TailwindCSS and supporting libraries such as lucide-react and date-fns. It provides role-aware screens for clients and mechanics. The MechanicBookingDetailPage is especially

important because it displays booking status, OBD entry, prediction results, and report actions.

The ML service is implemented with FastAPI. It hosts the Keras model and scaler and exposes a prediction endpoint used by Spring Boot. PostgreSQL runs on port 5433 in development. MailHog is used for email testing in the development environment.

Layer	Technology	Role
Frontend	React, TypeScript, TailwindCSS	Client and mechanic user interface.
Backend	Java 17, Spring Boot 3.2.3	Business logic, security, persistence, reports.
Machine Learning	Python, FastAPI, Keras, joblib/scikit-learn	Loads model and returns fault prediction.
Database	PostgreSQL	Stores users, vehicles, bookings, OBD data, reports, maintenance records.
PDF	iTextPDF	Generates booking diagnostic reports.
Development services	Docker Compose, MailHog	Containerized database/service support and email testing.

Data Transfer Objects and Validation

DTOs are small objects that define what data is accepted or returned by an API endpoint. They are useful because the database entity should not always be exposed directly to the outside world. For example, a user entity contains a password field, but an API response should never return the password.

For diagnostic submission, the OBDDataDTO requires the 14 numeric fields. This protects the ML service from incomplete input. If one value is missing, the backend can reject the request before calling FastAPI. This is cleaner than letting the Python service fail later with a vague error.

DTOs also make frontend integration easier when they are stable. A problem appeared when Java Records produced camelCase names while older frontend code expected snake_case names. The solution was to update the frontend and make the displayed values defensive against missing or invalid data.

Backend Module Breakdown

The Spring Boot backend is organized around feature modules. This structure improves maintainability because each module has a clear responsibility and can be tested independently. The backend is the authoritative system for permissions and persistence.

Module	Responsibility
auth	Registration, login, JWT generation, authentication response DTOs.
security	JWT filter, stateless security configuration, role-based route protection.
user	User entity, role management, duplicate email checks.
vehicle	Vehicle CRUD and ownership verification.
booking	Booking creation, listing, mechanic assignment, status transitions.
diagnostic	OBDData DTO validation, diagnostic persistence, ML result storage.
ml	FastAPI client and direct ML passthrough endpoint for testing.
report	iTextPDF generation, report metadata, file download.
maintenance	Maintenance record creation and retrieval.

Module	Responsibility
email	Booking confirmation and report-ready email triggers.

Frontend Module Breakdown

The frontend is designed around role-specific workflows. The client screens focus on vehicles, booking creation, and history. The mechanic screens focus on the work queue, booking details, OBD entry, prediction output, and completion actions.

A real frontend issue appeared when backend DTOs changed to Java Records. The frontend expected older field names and could not display fault information correctly. The fix was to read the new camelCase fields and keep safe fallbacks where required. This is a good example of why API contracts must be documented and kept consistent.

Frontend area	Main responsibility
Authentication pages	Login/register flow and token storage.
Client dashboard	Vehicles, bookings, diagnostic history, report access.
Mechanic dashboard	Pending and assigned bookings with status filters.
Mechanic booking detail	Status actions, OBD entry, prediction result, report generation.
Diagnostic result card	Displays predicted fault, confidence, visual severity, and safe missing-value handling.
API client	Centralizes REST calls to the Spring Boot backend.

Report Generation

Report generation is handled by the Spring Boot backend using iTextPDF. The report is linked to a booking, which ensures the generated file can include the relevant client, vehicle, mechanic, OBD, prediction, and maintenance data.

A complete diagnostic report should not only show the predicted fault. It should include the vehicle identity, booking date, OBD values, predicted label, confidence score, mechanic notes, parts replaced if any, cost if recorded, and recommended next actions. This makes the output useful for the client and defensible as a professional maintenance document.

Report section	Content
Header	Report title, booking ID, creation date.
Vehicle information	Make, model, year, VIN/license plate when available.
Diagnostic readings	The 14 OBD values submitted by the mechanic.
AI result	Predicted fault label and confidence score.
Maintenance information	Intervention, notes, parts replaced, and cost.
Conclusion	Mechanic summary and suggested follow-up.

Deployment and Development Environment

The development environment is designed to keep services separated while still allowing them to communicate locally. PostgreSQL is exposed on port 5433 to avoid conflicts with local databases. The FastAPI ML service runs on port 8001. The Spring Boot backend runs on port 8080. The frontend is expected on Vite or React development

ports such as 5173 or 3000, which are allowed by CORS configuration.

MailHog is used as a safe local email capture tool. This allows booking-confirmation and report-ready email flows to be tested without sending real emails during development.

Service	Typical port	Purpose
React frontend	5173 or 3000	User interface for clients and mechanics.
Spring Boot backend	8080	Main API, security, persistence, reports.
FastAPI ML service	8001	Prediction service using Keras model and scaler.
PostgreSQL	5433 mapped to 5432	Persistent relational database.
MailHog SMTP/UI	1025 / 8025	Development email capture and inspection.

API Summary

The following endpoints summarize the main contract implemented by the Spring Boot backend. They are presented as a technical appendix inside the main report because they are useful during presentation and defense.

Module	Endpoint	Purpose
Authentication	POST /api/auth/register, POST /api/auth/login	Create account and obtain JWT token.
Vehicles	GET /api/vehicles/my, POST /api/vehicles, PUT /api/vehicles/{id}, DELETE /api/vehicles/{id}	Manage vehicle records.
Bookings	POST /api/bookings, GET /api/bookings/my	Create and list bookings.
Bookings	POST /api/bookings/{id}/accept, /start, /complete	Move booking through the workflow.
Diagnostics	POST /api/diagnostic/booking/{bookingId}	Submit OBD values and store prediction.
Diagnostics	GET /api/diagnostic/booking/{bookingId}	Retrieve diagnostic result for a booking.
Reports	POST /api/reports/booking/{bookingId}, GET /api/reports/download/{reportId}	Generate and download PDF report.
Maintenance	POST /api/maintenance/booking/{bookingId}	Log intervention details and cost.
ML	POST /api/ml/predict	Direct passthrough test to FastAPI.

Limitations

The project is functional as a PFE prototype, but it still has limitations that should be stated clearly. Stating limitations is better than overstating the system because it shows engineering maturity and keeps the report credible.

Limitation	Explanation
Manual OBD entry	The current workflow assumes values are entered by the mechanic instead of streamed directly from an OBD-II adapter.
Label mismatch	FastAPI and Java use different human-readable fault descriptions for the same predicted class.
No production deployment proof	The project is documented mainly for local/containerized development and academic demonstration.

Limitation	Explanation
Model data dependency	Prediction quality depends on the representativeness and quality of the training dataset.
Frontend repository separation	The backend repository allows frontend origins, but the frontend may be managed separately from the backend source package.

Testing and Validation

Testing was documented as a Waterfall verification phase. The objective was to confirm that the delivered system matches the requirements and that critical workflows behave correctly. The report avoids invented coverage values. Instead, it lists the concrete validation scenarios that were used or should be demonstrated during the defense.

ID	Validation scenario	Expected result
T-01	Register and log in as CLIENT and MECHANIC.	JWT token returned and role-specific access applied.
T-02	Client creates a vehicle and then creates a booking.	Vehicle ownership is enforced and booking starts as PENDING.
T-03	Mechanic accepts and starts a booking.	Mechanic is assigned; status moves to CONFIRMED then IN_PROGRESS.
T-04	Assigned mechanic submits all 14 OBD values.	Values are validated, saved, and forwarded to FastAPI.
T-05	FastAPI returns predicted class and confidence.	Spring Boot stores predictedFault and confidenceScore.
T-06	Generate report for a completed/diagnosed booking.	PDF metadata is stored and file is available for download.
T-07	Unauthorized client attempts to access another client's vehicle.	Request is rejected by ownership guard.
T-08	Unassigned mechanic attempts diagnostic submission.	Request is rejected by service-layer authorization.

End-to-End Demonstration Scenario

For the final presentation, the best demonstration is an end-to-end scenario because it shows the full value of the system. The presenter can start as a client, create or select a vehicle, and request a diagnostic booking. This proves the client side and vehicle ownership logic.

Then the presenter switches to the mechanic role. The mechanic sees the pending booking, accepts it, and starts the diagnostic. This proves that the booking lifecycle is enforced and that mechanics have a real operational dashboard.

Next, the mechanic enters the 14 OBD values. The system sends the values to the ML service, receives a fault prediction and confidence score, and displays the result. This is the moment where the AI component is connected to the business workflow.

Finally, the mechanic completes the booking, adds maintenance notes, and generates the report. The client can then access the result. This final step proves traceability from appointment request to diagnostic conclusion.

Step	What to show	Why it matters
1	Client login and vehicle list.	Shows role-based client interface and ownership.

Step	What to show	Why it matters
2	Create booking for a selected vehicle.	Shows operational workflow starts from the client need.
3	Mechanic accepts and starts booking.	Shows booking state machine.
4	Mechanic submits OBD values.	Shows the diagnostic form and validation.
5	Prediction result appears.	Shows FastAPI/Keras integration.
6	Generate/download PDF report.	Shows final deliverable for the client.

Class Diagram Explanation in Words

A class diagram can be difficult for a non-technical reader, so this section explains the same idea in simple language. A class represents an important concept in the application. For example, User represents a person who can log in. Vehicle represents a car registered by a client. Booking represents a maintenance appointment. OBDData represents the diagnostic numbers and prediction result.

The User class has a role. This role is important because the same application serves two different types of people. A client and a mechanic are both users, but they do different actions. The backend checks this role before allowing protected operations.

The Vehicle class belongs to a client. This relationship allows the system to protect data ownership. If a vehicle is owned by one client, another client should not be able to create bookings for it or view its history.

The Booking class is the main coordination object. It connects the client, the mechanic, the vehicle, the scheduled time, and the current status. Because OBDData, Report, and MaintenanceRecord are linked to Booking, the system can reconstruct the full story of one maintenance intervention.

The OBDData class stores both the input and output of the diagnostic operation. The input is the list of 14 sensor values. The output is the predicted fault and confidence score. Keeping both together is useful because the result can always be traced back to the values that produced it.

Class	Simple explanation	Why it matters
User	A person who can access the system.	Controls authentication and role-based behavior.
Vehicle	A car registered by a client.	Connects bookings and diagnostic history to a real asset.
Booking	A scheduled maintenance or diagnostic appointment.	Coordinates the whole workflow.
OBDData	The sensor readings plus AI result.	Stores the evidence used for the prediction.
Report	The generated PDF file record.	Gives the client a readable final document.
MaintenanceRecord	The mechanic's intervention details.	Preserves what was done after diagnosis.

Sequence Workflows Explained

A sequence diagram shows how different parts of the system communicate over time. In this project, the two most important sequences are booking creation and diagnostic prediction.

For booking creation, the client sends a request from the frontend. The Spring Boot controller receives it, the service checks that the vehicle belongs to the client, and the repository saves the booking in PostgreSQL. The response returns the booking with PENDING status. This is a simple flow, but it proves that user interface, backend validation, and database persistence work together.

For diagnostic prediction, the mechanic submits the OBD form. The backend first checks that the mechanic is assigned to the booking and that the booking is in a valid state. Then it transforms the DTO into the request expected by FastAPI. FastAPI runs the scaler and neural network, returns the predicted class and confidence, and Spring Boot saves the result in the OBDData table.

The reason this sequence matters is that many errors can happen at the boundaries: wrong field names, wrong numeric format, missing values, wrong service URL, or unavailable ML service. The implementation must therefore be careful at every exchange point.

Workflow	Main participants	Critical check
Create booking	React frontend, BookingController, BookingService, PostgreSQL	Vehicle must belong to authenticated client.
Accept booking	Mechanic UI, BookingService, database	Booking must be PENDING and unassigned.
Start booking	Mechanic UI, BookingService	Mechanic must be the assigned mechanic.
Submit diagnostic	React form, DiagnosticService, FastApiClient, FastAPI	All 14 values must exist and booking must be valid.
Generate report	ReportController, ReportService, iTextPDF, file system	Booking must have enough diagnostic data to produce useful output.

Business Rules

Business rules are the rules of the real process that the software must respect. They are different from visual design choices. A button color can change without changing the system logic, but a booking should not move from PENDING directly to COMPLETED because that would break the real maintenance process.

The most important business rule is ownership. A client owns vehicles, and only that client should manage those vehicles. Another important rule is mechanic assignment. A mechanic can see pending work, but only the assigned mechanic should perform the diagnostic action for that booking.

The diagnostic rule is also strict. OBD data must be attached to a booking, not submitted as isolated information. This prevents disconnected predictions that cannot be explained later. If a client asks why a report says a certain fault was predicted, the system can point back to the exact booking and OBD values.

These business rules make the project stronger because they transform it from a basic CRUD application into a controlled maintenance platform.

Rule	Reason
A client can manage only owned vehicles.	Protects privacy and prevents incorrect bookings.
A booking starts as PENDING.	Shows that a mechanic has not accepted it yet.
Only a mechanic can accept/start/complete a booking.	Matches the real responsibility of workshop staff.
Only assigned mechanics can submit diagnostics.	Prevents unauthorized technical changes.
Diagnostic data is linked to a booking.	Keeps prediction results traceable.
Reports are linked to bookings.	Makes final documents easy to retrieve and audit.

Technical Glossary for the Defense

This glossary helps explain technical terms during the defense in simple language. It can also help a reader who is not familiar with software engineering understand what each term means in this project.

Term	Simple explanation
Frontend	The part of the application the user sees and interacts with in the browser.
Backend	The server-side part that applies rules, stores data, and protects the system.
REST API	A set of URLs used by the frontend to ask the backend for actions or data.
DTO	A safe data shape used to send or receive only the fields needed by an endpoint.
JWT	A login token that lets the backend know who is making a request.
JPA Repository	A Spring component that reads and writes database records.
Microservice	A small separate service that has one main responsibility.
FastAPI	The Python web service used to host the machine-learning prediction endpoint.
Keras model	The trained neural network used for fault classification.
Scaler	A preprocessing object that transforms raw numbers before prediction.
Confidence score	The model's strength of belief in the predicted class.
PDF report	The final client-readable document generated from booking and diagnostic data.

Defense Explanation Guide

During the presentation, the project should be explained as a complete workflow rather than only a set of technologies. A clear opening sentence could be: this platform helps a workshop manage diagnostic appointments and use OBD data with a neural network to support engine fault detection.

The second point should be the architecture. Explain that React is used for the interface, Spring Boot is used for secure business logic and data management, FastAPI is used for the machine-learning model, and PostgreSQL stores the persistent records.

The third point should be the booking lifecycle. This is where the project becomes realistic. The AI prediction happens inside a booking controlled by status transitions, not as a random standalone prediction.

The fourth point should be the ML integration. Keep it simple: the mechanic enters 14 OBD values, the backend validates them, FastAPI scales them and runs the neural network, then the backend stores the predicted fault and confidence.

The final point should be honesty about limitations and future work. Mention direct OBD hardware integration, label synchronization between Python and Java, cloud deployment, and larger real-world training data.

Presentation moment	Message to communicate
Opening	The project combines workshop management and AI-assisted diagnostics.
Architecture	Each layer has a clear job: UI, business logic, prediction, database.
Workflow	Booking status controls when diagnostics can happen.
AI part	The model supports the mechanic; it does not replace professional judgment.
Validation	The main scenario is tested from client booking to generated report.
Conclusion	The system is a strong prototype with clear future improvements.

Sample Report Content Expected From the Platform

The generated maintenance report is one of the final outputs of the application. It should be understandable for a client who is not technical, while still containing enough information for a mechanic or supervisor to verify the diagnostic result.

A good report should start with simple identification information: booking number, client, vehicle, mechanic, and date. Then it should show the diagnostic summary in normal language. For example, instead of only writing a class number, the report should say whether the system detected no fault, an ignition-related issue, or a fuel/emission-control issue.

The report should also include the confidence score, but the wording must be careful. It should not say that the system is certainly correct. A better sentence is: the model classified the submitted OBD pattern as this fault category with this confidence score. This keeps the mechanic's role clear.

Finally, the report should include the mechanic's notes, maintenance intervention, parts replaced, cost if available, and future recommendations. This turns the report into a useful maintenance document rather than a simple AI output screenshot.

Report field	Example wording
Vehicle	2018 Toyota Corolla, client vehicle registered in the platform.
Diagnostic input	14 OBD readings were submitted during the active booking.
AI result	Predicted fault: Fuel System/Emission Control Fault.
Confidence	The model returned a confidence score stored with the diagnostic result.
Mechanic notes	Observed rough idle; inspection recommended for spark plugs and fuel mixture.
Recommendation	Perform physical verification and schedule follow-up maintenance if symptoms persist.

Problems Faced and Solutions

Several concrete implementation issues were identified from the project notes. They are included here because they make the report more honest and technically credible.

Problem	Impact	Solution or handling
Java Records changed JSON field names to camelCase.	Frontend components expected older snake_case names and displayed missing values.	Updated frontend mapping to read faultDescription/predictedFault style fields and added fallbacks where needed.
Confidence sometimes displayed as NaN%.	Prediction result card looked broken when confidence was absent or differently scaled.	Added defensive formatting: show a dash for missing values and handle decimal or percentage inputs.
FastAPI faultDescription is not the same string stored by Java.	Reports and frontend may show Java-side labels instead of Python descriptions.	Documented the mapping clearly; future fix is to persist prediction.faultDescription() if more precise labels are required.
Docker and service routing differ between local and container execution.	Spring Boot, PostgreSQL, and FastAPI connectivity can break if hostnames or ports are wrong.	Centralized service URLs in configuration and used Docker Compose for repeatable development.
Legacy diagnostic ML client exists beside the newer FastApiClient.	It can confuse maintainers because it uses a different schema.	Kept the authoritative flow documented around Spring RestClient and the FastAPI service.

Problem	Impact	Solution or handling
Report generation originally risked being too minimal.	A PDF with only prediction text is not useful for a client.	The report structure should include vehicle info, OBD values, prediction, confidence, mechanic notes, and maintenance record.

Conclusion and Future Work

The project delivered a coherent platform that connects booking management with AI-assisted vehicle diagnostics. Its value is not only in the neural network, but in the workflow around it: authentication, vehicle ownership, mechanic assignment, controlled diagnostic submission, prediction storage, maintenance logging, and PDF reporting.

The Waterfall presentation makes the project easier to defend academically because each part can be traced from requirement to design and implementation. The final system demonstrates a practical multi-service architecture using Spring Boot for business logic, FastAPI for inference, PostgreSQL for persistence, and React/TypeScript for the interface.

Future improvements include direct OBD-II hardware acquisition instead of manual input, better synchronization of Python and Java fault labels, deployment to a cloud environment, improved report templates, dashboard analytics for mechanic workload and fault rates, and expanded model training with larger real-world datasets.

References

Chollet, F. Keras documentation. <https://keras.io>

FastAPI documentation. <https://fastapi.tiangolo.com>

Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>

PostgreSQL Documentation. <https://www.postgresql.org/docs/>

ISO 15031-5: Road vehicles - Communication between vehicle and external equipment for emissions-related diagnostics.

Mobley, R. K. An Introduction to Predictive Maintenance, 2nd edition, Butterworth-Heinemann.